

EMPLOYEE ATTRITION PREDICTION

Developer's MVP Definition

Backend Development Backlog, Responsibilities, Roadmap & Definition of Done

1. Developer Vision

As the Backend Developer, my objective is to build a secure, scalable, and modular backend system that powers the Employee Attrition Prediction platform. The backend provides REST APIs for authentication, employee management, machine learning prediction, dashboard analytics, reporting, and seamless integration with the frontend, enabling HR teams to make data-driven retention decisions.

- **Backend Developer:** Shiva Shankar
- **Delivery Window:** 02 July 2026 – 14 July 2026 (5 Sprints)
- **Target Release:** MVP Demo at the end of Sprint 5 (14-07-2026)

2. Target Users / Personas

Persona	Role	Backend Need	Key Backend Interaction
Priya	HR Business Partner	Securely access employee attrition predictions	Login, view dashboard statistics, fetch employee predictions and reports
Rahul	HR Analyst	Manage employee records and perform bulk predictions	CRUD operations, batch prediction APIs, report generation APIs

Persona	Role	Backend Need	Key Backend Interaction
Meera	HR Director	Monitor overall attrition trends and analytics	Dashboard APIs, prediction history, analytics reports

3. Problem Statement & Developer MVP Goal

Problem:

A machine learning model alone is not sufficient to solve employee attrition prediction. HR users require a secure backend that manages employee records, processes prediction requests, stores prediction history, and provides analytics through REST APIs for frontend consumption.

Developer MVP Goal:

Develop a complete backend application that exposes secure REST APIs for employee management, authentication, machine learning prediction, dashboard analytics, reporting, and data processing. The backend should integrate seamlessly with the React frontend and support an end-to-end employee attrition prediction workflow.

Success Metrics (Developer View):

- All REST APIs function correctly and are documented using Swagger.
- JWT authentication protects secured endpoints.
- Employee CRUD operations work successfully.
- Machine learning APIs generate accurate predictions.
- Dashboard APIs provide real-time analytics data.
- Reports and prediction history are retrievable through APIs.
- Backend integrates successfully with the React frontend.
- Complete workflow executes without manual intervention.

4. Developer Backlog (MoSCoW Prioritization)

Priority	Backend Module	Developer User Story
Must Have	FastAPI Project Setup	As a developer, I need a modular backend project so future development remains maintainable.
Must Have	Database Integration	As a developer, I need SQLAlchemy models and database connectivity to store employee and prediction data.
Must Have	Authentication (JWT)	As a developer, I need secure authentication so only authorized users access the system.
Must Have	Employee CRUD APIs	As an HR Analyst, I need to create, update, view, and delete employee records.
Must Have	Dataset Processing APIs	As an ML pipeline, I need preprocessing APIs so training uses clean data.
Must Have	Model Training API	As a developer, I need APIs to train and evaluate machine learning models.
Must Have	Prediction APIs	As an HR user, I need single and batch predictions through REST APIs.

Priority	Backend Module	Developer User Story
Must Have	Dashboard APIs	As an HR Director, I need dashboard statistics and analytics.
Must Have	Prediction History	As an HR Analyst, I need prediction records stored for auditing and reporting.
Must Have	Reports APIs	As an HR Analyst, I need employee, prediction, and training reports.
Should Have	Analytics APIs	As HR Management, I need additional analytics for better decision-making.
Should Have	Logging	As a developer, I need application logs for debugging and monitoring.
Won't Have (Post-MVP)	HRMS Integration	Deferred for future releases.
Won't Have (Post-MVP)	Email Notification Service	Deferred after MVP validation.
Won't Have (Post-MVP)	Multi-Organization Support	Deferred until production release.
Won't Have (Post-MVP)	Advanced RBAC	Basic JWT authentication is sufficient for MVP.

Priority	Backend Module	Developer User Story
Won't Have (Post-MVP)	Mobile Backend APIs	Web application support only in MVP.

5. Release Roadmap (Sprint View)

Sprint	Date	Developer Focus	Key Deliverable
Sprint 1	02-07-2026	Project setup, database configuration, FastAPI initialization, Swagger documentation	Backend project structure, database models, API documentation
Sprint 2	05-07-2026	Authentication module and Employee CRUD implementation	JWT Authentication, User APIs, Employee Management APIs
Sprint 3	08-07-2026	Machine Learning integration, dataset processing, prediction APIs	Training API, Prediction APIs, Batch Prediction, Model Metrics
Sprint 4	11-07-2026	Dashboard APIs, Prediction History, Reports APIs, Backend refactoring	Dashboard Module, Reports Module, Modular API Architecture

Sprint	Date	Developer Focus	Key Deliverable
Sprint 5	14-07-2026	Backend testing, frontend integration support, API validation, documentation	Fully integrated backend ready for MVP demonstration

6. Definition of Done (Developer MVP)

The backend MVP is considered complete when:

- REST APIs are implemented using FastAPI.
- JWT Authentication is functional.
- Employee CRUD operations work end-to-end.
- Dataset preprocessing and feature engineering APIs are available.
- Machine learning model training API executes successfully.
- Prediction APIs support single and batch predictions.
- Dashboard APIs provide statistics and analytics.
- Prediction history is stored in the database.
- Reports APIs return employee, prediction, and training reports.
- Backend routes are modularized into separate route files.
- Swagger documentation is complete and functional.
- Backend integrates successfully with the React frontend.
- End-to-end workflow (Login → Employee Management → Prediction → Dashboard → Reports) executes successfully.
- Backend is successfully demonstrated during the Sprint 5 MVP presentation.

7. Key Risks & Developer Mitigation

Risk	Impact on Backend	Mitigation
Tight 5-sprint timeline	Limited testing time	Prioritize core APIs first,

Risk	Impact on Backend	Mitigation
		complete refactoring after functionality is stable
Poor dataset quality	Reduced prediction accuracy	Implement preprocessing, validation, and feature engineering before training
Authentication vulnerabilities	Unauthorized system access	Implement JWT authentication and secure protected APIs
Backend complexity	Difficult maintenance	Modularize routes, services, schemas, and database layers
Frontend integration issues	UI unable to consume APIs	Maintain REST API standards, test endpoints through Swagger before integration
Machine learning performance	Slow predictions	Optimize preprocessing pipeline and reuse trained models where appropriate

8. Stakeholders

- **Product Owner:** Akshay — Defines product vision, prioritizes backlog, approves MVP scope.
- **Backend Developer:** Shiva Shankar — Designs, develops, tests, and integrates backend APIs and machine learning services.
- **Frontend Developer:** Responsible for building the React-based user interface and

integrating backend APIs.

- **Machine Learning Team:** Develops and validates prediction models (Logistic Regression, Random Forest, XGBoost, SHAP).
- **End Users:** HR Business Partners, HR Analysts, HR Directors.
- **Project Evaluators:** Final Year Project – AI/ML Track Review Committee.